Runtime Optimizations for the Mondrian Forest Implementation

Nina Li

July 12, 2026

1 Summary

After tracing how the code grows the trees and computes the predictions, I found three places that seem to involve a large amount of unnecessary computation. These include maintaining information that is never used later and using general-purpose methods that do more work than necessary to compute quantities that can be obtained more directly. I therefore modified these parts of the code to remove or simplify the unnecessary computation.

In particular, the changes do not affect how the Mondrian trees are generated. The sampled cut times, cut dimensions, cut locations, tree structures, and assignments of points to leaves all remain the same. The changes only simplify how this information is stored and how the final leaf values are calculated.

I then ran the original and optimized versions using the same random seeds. All recorded MSE values and EGOP angle distances remained unchanged, while the runtime decreased substantially. I also went back through the TrIM paper to check whether the parts I changed were needed for the theoretical results. My understanding is that the theory depends on the Mondrian partitions and the resulting forest estimator, both of which remain unchanged, rather than on the specific way the code stores intermediate information or carries out the computation.

For each change below, I explain what the original code was doing, why I think the computation can be removed or simplified, and why the predictions and theoretical results remain unchanged after the optimization. However, since some of these changes remove parts of the original implementation or replace the original computation with an equivalent one, I would still appreciate your confirmation that I have not missed any reason they need to be kept.

2 Background: How the Code Implements the Mondrian Forest

The implementation uses a sparse matrix to record the leaf assignment of every data point in each tree. Each row corresponds to one data point, and the columns correspond to the leaves created across all M trees. Since each point belongs to exactly one leaf in each tree, every row contains exactly M nonzero entries.

When a leaf is split, the code divides the points in that leaf between its two child leaves and

updates their column indices in the sparse matrix. This process continues in the order determined by the Mondrian process until the maximum lifetime is reached. The sparse matrix is then used to calculate the leaf values and the predictions.

The three optimizations are about:

(1) how the leaf values are calculated

(2) a list that is repeatedly updated but never used

(3) how often the sparse matrix is reconstructed

3 Optimization 1: Replace the Dense Linear Solve with Direct Division

3.1 What the Original Code Does

After a tree has been grown, the code needs to compute one prediction value for each active leaf. Equation (3) of the TrIM paper defines the prediction from one tree as

$$\widehat{f}_n(x, \mathcal{P}) = \sum_{i=1}^{n} \frac{\mathbf{1}_{\{X_i \in Z_x\}}}{N_n(x)} Y_i,$$

where Z_x is the leaf containing x and $N_n(x)$ is the number of training points in that leaf.

The code implementation does not compute the within-leaf average in Equation (3) by directly summing the responses and dividing by the number of points in the leaf. Instead, it first centers the training responses:

$$y_{\text{train},i} = y_i - \bar{y}.$$

For each tree, it then constructs a leaf-membership matrix Z_{train} and solves for the leaf-weight vector w:

$$\left(Z_{\text{train}}^\top Z_{\text{train}} + \frac{\delta}{M} I\right) w = Z_{\text{train}}^\top y_{\text{train}}.$$

Here, w contains one internal weight for each active leaf, M is the number of trees, and $\delta \geq 0$ is a regularization parameter. The original implementation solves this system using `np.linalg.solve`.

The weights w are not themselves the final leaf predictions. For a test point, the code first computes $Z_{\text{test}} w$ and then adds back the overall response mean $\bar{y}$. With $\delta = 0$ (the setting in our experiments), the resulting prediction is exactly the within-leaf average defined in Equation (3).

For a tree with L active leaves, solving this system as a general dense linear system requires $O(L^3)$ time and $O(L^2)$ memory.

3.2 Simplification

The original code solves

$$\left(Z_{\text{train}}^\top Z_{\text{train}} + \frac{\delta}{M} I\right) w = Z_{\text{train}}^\top y_{\text{train}}.$$

Although this is written as a general linear system, its coefficient matrix is diagonal.

For one tree, each training point belongs to exactly one active leaf. Therefore, each row of Z_{train} contains one nonzero entry, equal to $1/\sqrt{M}$. Two different leaf columns can never be nonzero in the same row, so their inner product is zero. This means that all off-diagonal entries of $Z_{\text{train}}^\top Z_{\text{train}}$ are zero.

If leaf ℓ contains n_ℓ training points, its diagonal entry is

$$\left(Z_{\text{train}}^\top Z_{\text{train}}\right)_{\ell\ell} = n_\ell \left(\frac{1}{\sqrt{M}}\right)^2 = \frac{n_\ell}{M}.$$

After including the regularization term, the coefficient matrix becomes

$$Z_{\text{train}}^\top Z_{\text{train}} + \frac{\delta}{M} I = \frac{1}{M} \operatorname{diag}\left(n_1 + \delta, \ldots, n_L + \delta\right).$$

The linear system therefore separates into L independent scalar equations. For each leaf ℓ,

$$\frac{n_\ell + \delta}{M} w_\ell = \left(Z_{\text{train}}^\top y_{\text{train}}\right)_\ell,$$

so its weight can be computed directly as

$$w_\ell = \frac{\left(Z_{\text{train}}^\top y_{\text{train}}\right)_\ell}{(n_\ell + \delta)/M}.$$

Thus, the code does not need to construct and solve a general dense linear system. It only needs to compute the diagonal entries and the right-hand side, and then divide them elementwise. This gives exactly the same solution as `np.linalg.solve` because the system is diagonal.

This direct calculation requires $O(N+L)$ time: Computing the diagonal entries and the right-hand side requires one pass through the N training points, and solving for the weights requires one division for each of the L active leaves.

3.3 Implementation Change

The original implementation passes the system to a general dense linear solver:

Original code

```
Z_train = Z_all[:, active_features_in_tree[m]]

w_tree = np.linalg.solve(
    np.transpose(Z_train).dot(Z_train)
    + delta / M
    * np.identity(len(active_features_in_tree[m])),
    np.transpose(Z_train).dot(y_train)
)
```

The modified implementation extracts the diagonal entries of the same coefficient matrix and solves the system by element-wise division:

Modified code

```
Z_train = Z_all[:, active_features_in_tree[m]]
w_tree = _leaf_weights(Z_train, y_train, M, delta)

def _leaf_weights(Z_train, y_train, M, delta):
    # Diagonal of Z_train.T @ Z_train,
    # equal to diag(leaf counts) / M
    d = np.asarray(
        Z_train.multiply(Z_train).sum(axis=0)
    ).ravel() + delta / M

    b = np.asarray(
        np.transpose(Z_train).dot(y_train)
    ).ravel()

    return b / d
```

Here, `d` contains the diagonal entries of

$$Z_{\text{train}}^{\top} Z_{\text{train}} + \frac{\delta}{M} I,$$

and `b` is $Z_{\text{train}}^{\top} y_{\text{train}}$. Therefore, `b / d` gives the exact solution of the same diagonal system.

3.4 Evaluation and Why the Original Code Uses a General Solver

I evaluated this change using the full rotated experiment with $d = 30$, ACTIVE $= 2$, $M = 30$, and $\lambda = 5$. The original implementation and the diagonal-division version were run using the same seeds.

All recorded quantities that depend on the fitted estimator remained unchanged. The estimated EGOP angle distances were exactly the same, all MSE values across the different methods and lifetimes also matched exactly in the two runs. This is consistent with the calculation above, since the two implementations solve the same diagonal system. This also confirms that the change did not introduce an error in the implementation.

The runtime of the downstream Mondrian evaluation decreased substantially:

Downstream run	Original (s)	Diagonal division (s)	Speed-up
TrIM, $n = 3200$	295.45	42.22	7.0×
TrIM, $n = 6400$	1641.97	178.23	9.2×

Table 1: Runtime of the downstream Mondrian evaluation before and after replacing the dense solve with direct diagonal division.

During the downstream lifetime sweep, the code recalculates the leaf weights at every evaluation checkpoint, which occurs once every 50 split events. The system is therefore solved repeatedly as the forest grows. As the number of active leaves increases, repeatedly applying an $O(L^3)$ dense solve becomes expensive. Replacing it with the $O(N + L)$ direct calculation therefore has a larger effect on the downstream evaluation.

Finally, the reason why the original implementation uses a general linear solver rather than direct division is that according to the docstring (explanation of the code written by author), TrIM Mondrian code is based on Matej Balog's Mondrian-kernel implementation, which was written for a more general setting that supported both Mondrian kernels and Mondrian forests.

In the Mondrian-kernel setting, the leaf features from all M trees are concatenated into a single design matrix, and a ridge-regression problem is solved over this joint feature space. In the kernel setting, each row of the joint design matrix has M nonzero entries, so leaf columns belonging to different trees are simultaneously nonzero in that row. Their inner products are therefore generally nonzero, so $Z^\top Z$ is generally not diagonal, therefore a general linear solver is more appropriate.

The TrIM paper, however, chooses Mondrian forest estimators rather than kernel estimators (according to my understanding). Therefore the trees are fitted separately, and the leaf-weight system is solved one tree at a time. Restricting the design matrix to the leaves of a single tree makes each row contain only one nonzero entry, so $Z_{\text{train}}^\top Z_{\text{train}}$ is diagonal.

Therefore, the general linear solve became unnecessary once the computation was restricted to the Mondrian forest used by TrIM. My modification only computes the same Mondrian forest estimator more efficiently by using the diagonal structure of the forest-specific system.

4 Optimization 2: Remove an Unused Bookkeeping List

4.1 What the Original Code Does

The implementation maintains two different records of the currently active leaves:

- `active_features_in_tree[m]` stores the active leaves of tree m;
- `active_features` stores the active leaves from all M trees in one combined list.

Whenever a leaf is split, the code updates both records. It removes the old leaf and adds the two newly created child leaves to both the per-tree list and the combined list.

4.2 Simplification

The calculations used to obtain the forest predictions only read `active_features_in_tree[m]`. This per-tree list is used to select the active columns of the leaf-membership matrix for each tree.

In contrast, `active_features` is repeatedly modified but is never read anywhere in the current training or evaluation functions. I checked every occurrence of this variable: the code initializes the list and updates it after every split, but no later calculation uses it.

Maintaining the unused global list has a nontrivial computational cost. Initially, `active_features` contains one active leaf from each of the M trees. Each split removes one leaf and adds two children, so after s splits the list contains $M + s$ entries.

The operation `active_features.remove(c)` must search through the Python list to locate c, and therefore requires $O(M+s)$ time at the s-th split. If the forest performs S splits in total, maintaining

this unused list requires

$$\sum_{s=0}^{S-1} O(M+s) = O(SM + S^2)$$

time. For fixed M, this is $O(S^2)$. The two corresponding `append` operations have only amortized $O(1)$ cost per split, so the repeated linear search in `remove` is the dominant unnecessary cost.

4.3 Implementation Change

The original implementation updates both the global list and the per-tree list:

Original code

```
active_features = []

for m in range(M):
    active_features.append(m)

# After splitting leaf c:
active_features.remove(c)
active_features_in_tree[m].remove(c)

active_features.append(C + 0)
active_features.append(C + 1)

active_features_in_tree[m].append(C + 0)
active_features_in_tree[m].append(C + 1)
```

The modified implementation removes the unused global list and keeps only the per-tree updates:

Modified code

```
# After splitting leaf c:
active_features_in_tree[m].remove(c)
active_features_in_tree[m].append(C + 0)
active_features_in_tree[m].append(C + 1)
```

4.4 Evaluation and Why the Original Code Keeps a Unused List

I evaluated this change using the full rotated experiment with $d = 30$, ACTIVE $= 2$, $M = 30$, and $\lambda = 5$. The versions with and without the unused global list were run using the same seeds.

As expected, all recorded quantities that depend on the fitted estimator remained unchanged. The estimated EGOP angle distances were the same. All recorded MSE values also matched exactly.

Unlike Optimization 1, whose main runtime benefit appeared only in the downstream evaluation, removing the unused list accelerates both stages of the pipeline, because the list was maintained inside the tree-growth loop used by both `train` during EGOP estimation and `evaluate_all_lifetimes` during the downstream lifetime sweep.

Stage	Original (s)	Before Opt. 2 (s)	After Opts. 2 (s)	Cumulative speed-up
EGOP estimation, n=3200	1847.70	1847.70	269.00	6.9×
EGOP estimation, n=6400	7458.63	7458.63	734.29	10.2×
TrIM, $n = 3200$	295.45	42.22	10.58	27.9×
TrIM, $n = 6400$	1641.97	178.23	27.49	59.7×

Table 2: Cumulative downstream runtime improvement from the first two optimizations.

The runtimes before Optimization 2 already include the diagonal-division change from Optimization 1. Removing the unused global list provides a further 4.0× speed-up for $n = 3200$ and a further 6.5× speed-up for $n = 6400$. Relative to the initial, unoptimized implementation, the two changes together reduce the downstream runtime by approximately 27.9× and 59.7×, respectively.

The linear search removed in this optimization was performed after every split event. Since both estimating EGOP and downstream training grow Mondrian forests using the same basic growth loop, both repeatedly incurred this cost. Removing the unused list therefore reduces the runtime of both EGOP estimation and downstream evaluation.

The unused list also appears to be left over from the more general Mondrian-kernel implementation on which the TrIM code is based. In the Mondrian-kernel setting, the leaf features from all trees are combined into one feature map, so a global list of active leaf columns might be useful. TrIM, however, uses the Mondrian forest calculation, in which the trees are fitted separately. This requires only the per-tree lists `active_features_in_tree[m]`.

5 Optimization 3: Reconstruct the Sparse Matrix Only When It Is Needed

5.1 What the Original Code Does

The implementation uses a sparse matrix Z_{all} to record the current leaf assignment of every data point across all trees. When a leaf is split, the points in that leaf are reassigned to one of its two child leaves.

After every split, the original code performs two operations. First, it updates the stored leaf labels of the affected points. This step is necessary because it records their new leaf assignments. Second, it reconstructs the entire sparse matrix so that its number of columns matches the current number of leaves.

But this sparse matrix is not used to calculate predictions after every split. In `evaluate_all_lifetimes`, it is only read once every 50 split events, when an intermediate prediction error is evaluated.

5.2 Simplification

The leaf assignments must still be updated after every split, but the sparse matrix only needs to be reconstructed when it is about to be used.

In `evaluate_all_lifetimes`, the matrix is read only once every 50 split events, but the original code reconstructed the matrix after every split, so between two checkpoints, 49 reconstructed versions were created but never read.

The modified implementation continues to update `Z_all.indices` after every split, but only reconstruct matrix until immediately before an evaluation checkpoint. This change does not alter any leaf assignment. At each checkpoint, the matrix is reconstructed from the same updated assignments as in the original implementation. The skipped intermediate versions were never used, so the recorded errors, leaf weights, predictions, and MSE values remain unchanged.

5.3 Implementation Change

The original implementation updates the leaf assignments and then reconstructs the sparse matrix after every split:

Original code

```
# Update the leaf assignments of the moved points
Z_all.indices[feature_l * M + m] = C + 0
Z_all.indices[feature_r * M + m] = C + 1

# Reconstruct the CSR matrix after every split
Z_all = scipy.sparse.csr_matrix(
    (Z_all.data, Z_all.indices, Z_all.indptr),
    shape=(N_all, C + 2),
    copy=False
)

# The matrix is only read at an evaluation checkpoint
Z_train = Z_all[:N, active_features_in_tree[m]]
Z_test = Z_all[N:, active_features_in_tree[m]]
```

The modified implementation updates the leaf assignments after every split but reconstructs the matrix only when an evaluation is about to be performed:

Modified code

```
# Update the leaf assignments of the moved points
Z_all.indices[feature_l * M + m] = C + 0
Z_all.indices[feature_r * M + m] = C + 1

# Reconstruct the matrix only when it is about to be read
if counter % 50 == 0:
    Z_all = scipy.sparse.csr_matrix(
        (Z_all.data, Z_all.indices, Z_all.indptr),
        shape=(N_all, C),
        copy=False
    )

    Z_train = Z_all[:N, active_features_in_tree[m]]
    Z_test = Z_all[N:, active_features_in_tree[m]]
```

5.4 Evaluation

I evaluated this change using the same rotated experiment with $d = 30$, ACTIVE $= 2$, $M = 30$, and $\lambda = 5$. The versions with and without the per-split reconstruction were run using identical random seeds. The runtimes before Optimization 3 already include the improvements from Optimizations 1 and 2.

All recorded quantities that depend on the fitted estimator remained unchanged. The estimated EGOP angle distances were identical: 1.117027 for $n = 3200$ and 0.442698 for $n = 6400$. All recorded MSE values also matched exactly.

As in Optimization 2, this change accelerates both stages of the pipeline. The unnecessary reconstruction occurred inside the tree-growth loop used by both `train` for EGOP estimation and `evaluate_all_lifetimes` for downstream evaluation.

Stage	Original (s)	Before Opt. 3 (s)	After Opt. 3 (s)	Cumulative speed-up
EGOP estimation, $n = 3200$	1847.70	269.00	158.00	11.7×
EGOP estimation, $n = 6400$	7458.63	734.29	499.32	14.9×
TrIM, $n = 3200$	295.45	10.58	8.02	36.8×
TrIM, $n = 6400$	1641.97	27.49	22.38	73.4×

Table 3: Cumulative runtime improvement after all three optimizations.

Since the reconstruction was performed after every split, postponing it provides an additional speed-up in both EGOP estimation and downstream evaluation. The measured improvement is smaller than for the first two optimizations. This is expected because Optimization 3 removes a less expensive remaining operation after the dense solves and the repeated global-list searches have already been eliminated.

The runtimes before Optimization 3 already include Optimizations 1 and 2. Optimization 3 provides a further 1.70× speed-up for EGOP estimation for $n = 3200$, 1.47× speed-up for EGOP estimation for $n = 6400$, 1.32× for the downstream run with $n = 3200$, and 1.23× for the downstream run with $n = 6400$.

6 Conclusion

The three optimizations described above are implementation-level changes. They modify only how the program stores intermediate information and computes the leaf values, while leaving the Mondrian sampling process, tree structures, leaf assignments, and resulting forest estimator unchanged. Consistent with this, all recorded MSE values and EGOP angle distances matched before and after the changes across every tested configuration. The theoretical guarantees in the TrIM paper, which concern the forest estimator rather than its particular implementation, are therefore unaffected.

With the estimator held fixed, the runtime improvements were substantial. For the EGOP-estimation stage with $d = 30$ and ACTIVE $= 2$, the runtime decreased from around 31 minutes to around 2 minutes at $n = 3200$, corresponding to a 11.7× speed-up. At $n = 6400$, it decreased from around 2 hours to about 8 minutes, corresponding to a 14.9× speed-up.

For the downstream Mondrian evaluation, the full-transform TrIM runtime decreased from 295.45 seconds to 8.02 seconds at $n = 3200$, a 36.8× speed-up. At $n = 6400$, it decreased from 1641.97 seconds to 22.38 seconds, a 73.4× speed-up.